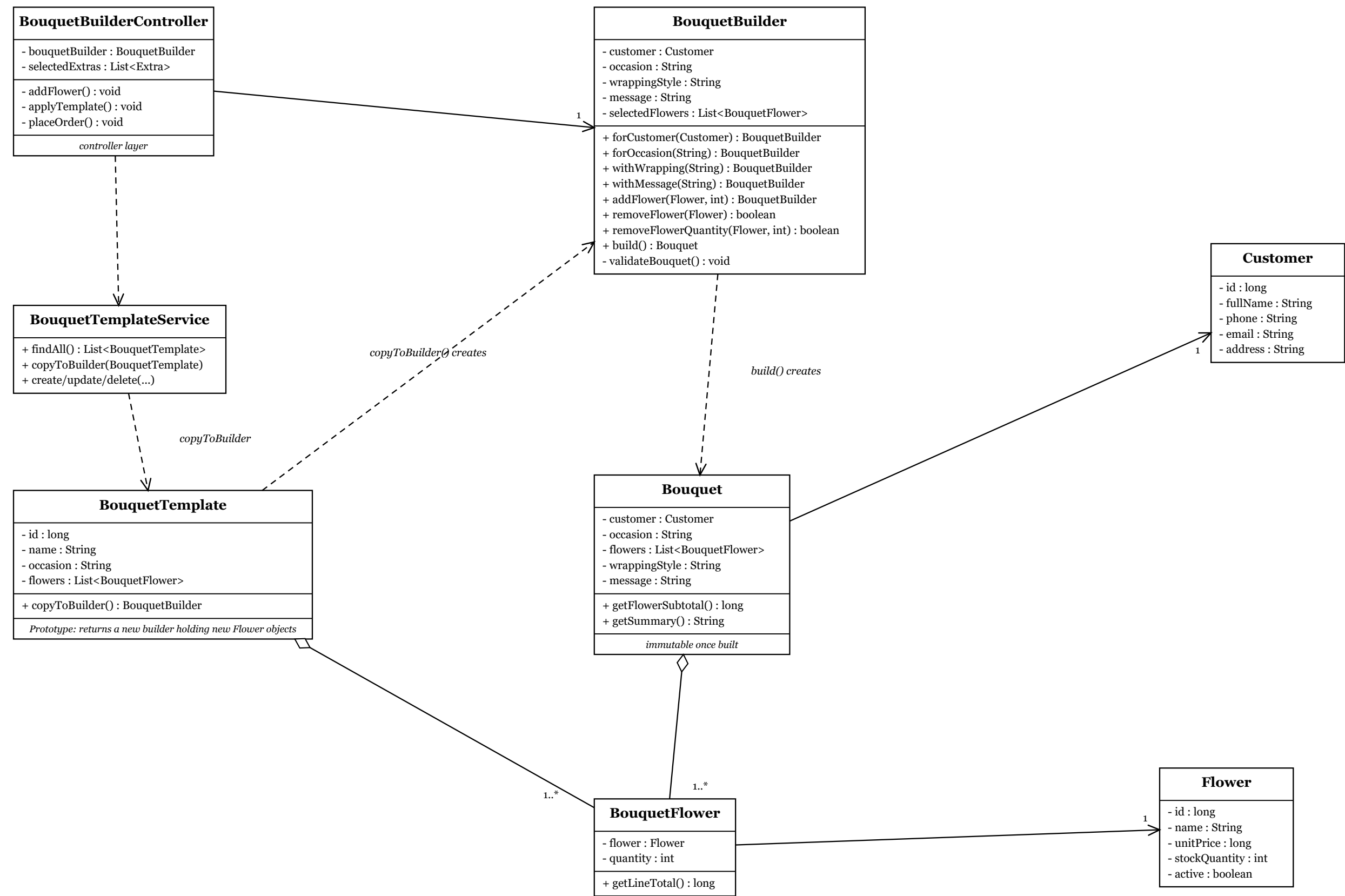


1. Builder and Prototype — constructing a bouquet

A bouquet is assembled over many UI events; a template is copied into an independent builder

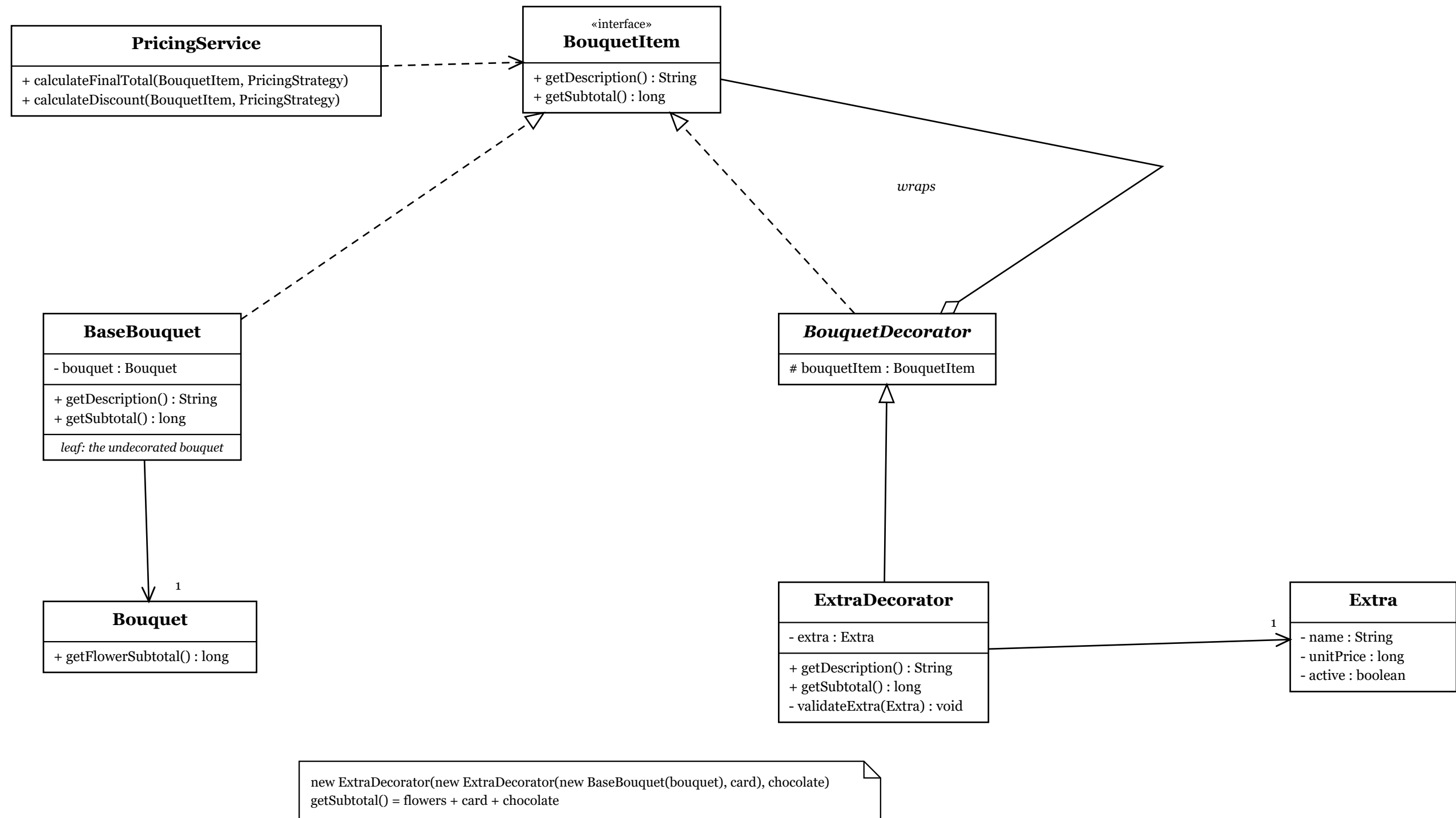


Notation:

solid line = association · open arrowhead = navigable · hollow diamond = aggregation
dashed line + hollow triangle = implements · solid line + hollow triangle = extends
dashed line + open arrowhead = uses (dependency) · italic type name = abstract

2. Decorator — optional bouquet extras

Each extra wraps a BouquetItem and adds to both the description and the subtotal



Notation:

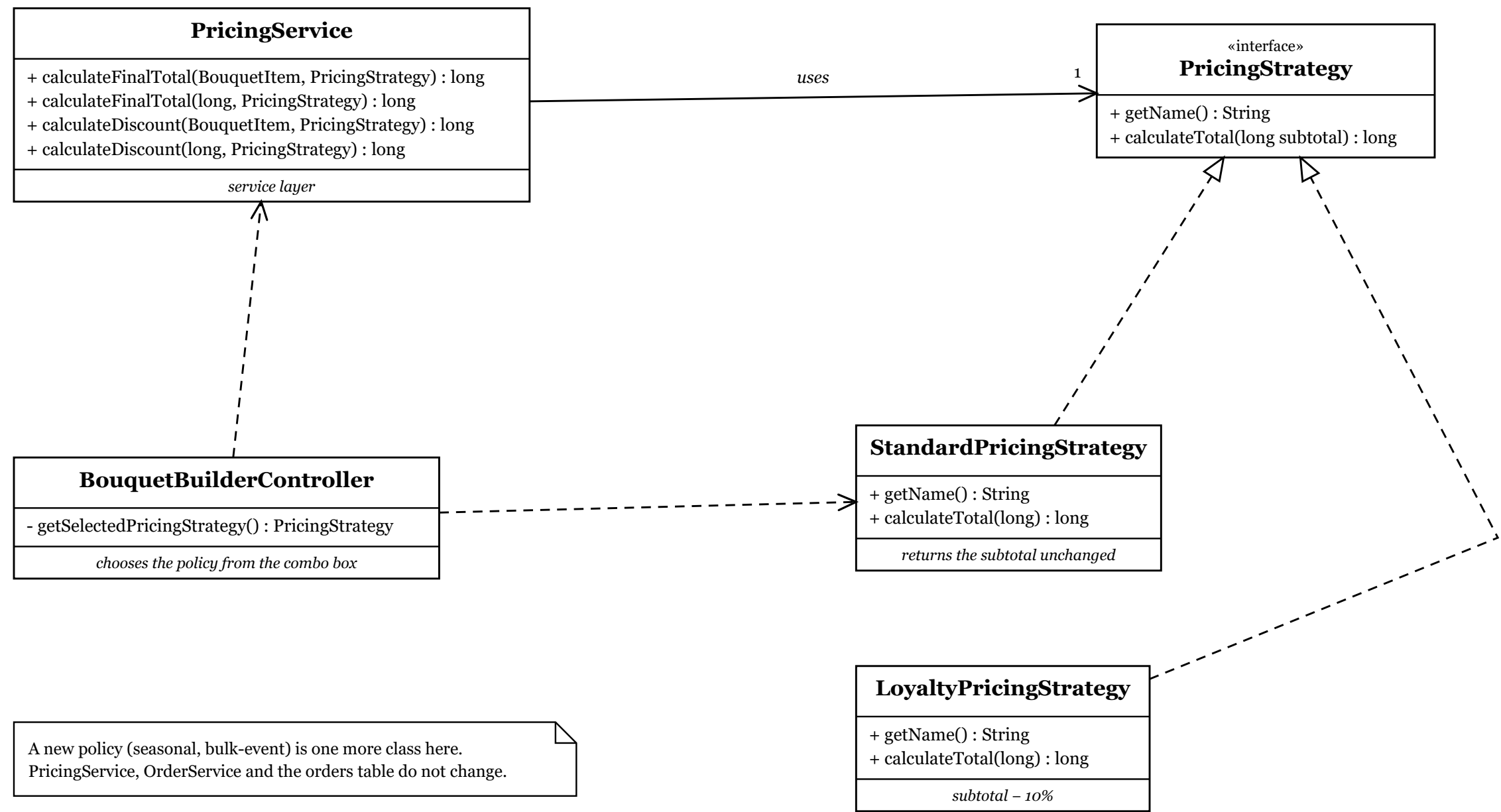
solid line = association · open arrowhead = navigable · hollow diamond = aggregation

dashed line + hollow triangle = implements · solid line + hollow triangle = extends

dashed line + open arrowhead = uses (dependency) · italic type name = abstract

3. Strategy — interchangeable pricing policies

Checkout depends on the PricingStrategy interface, never on a concrete policy

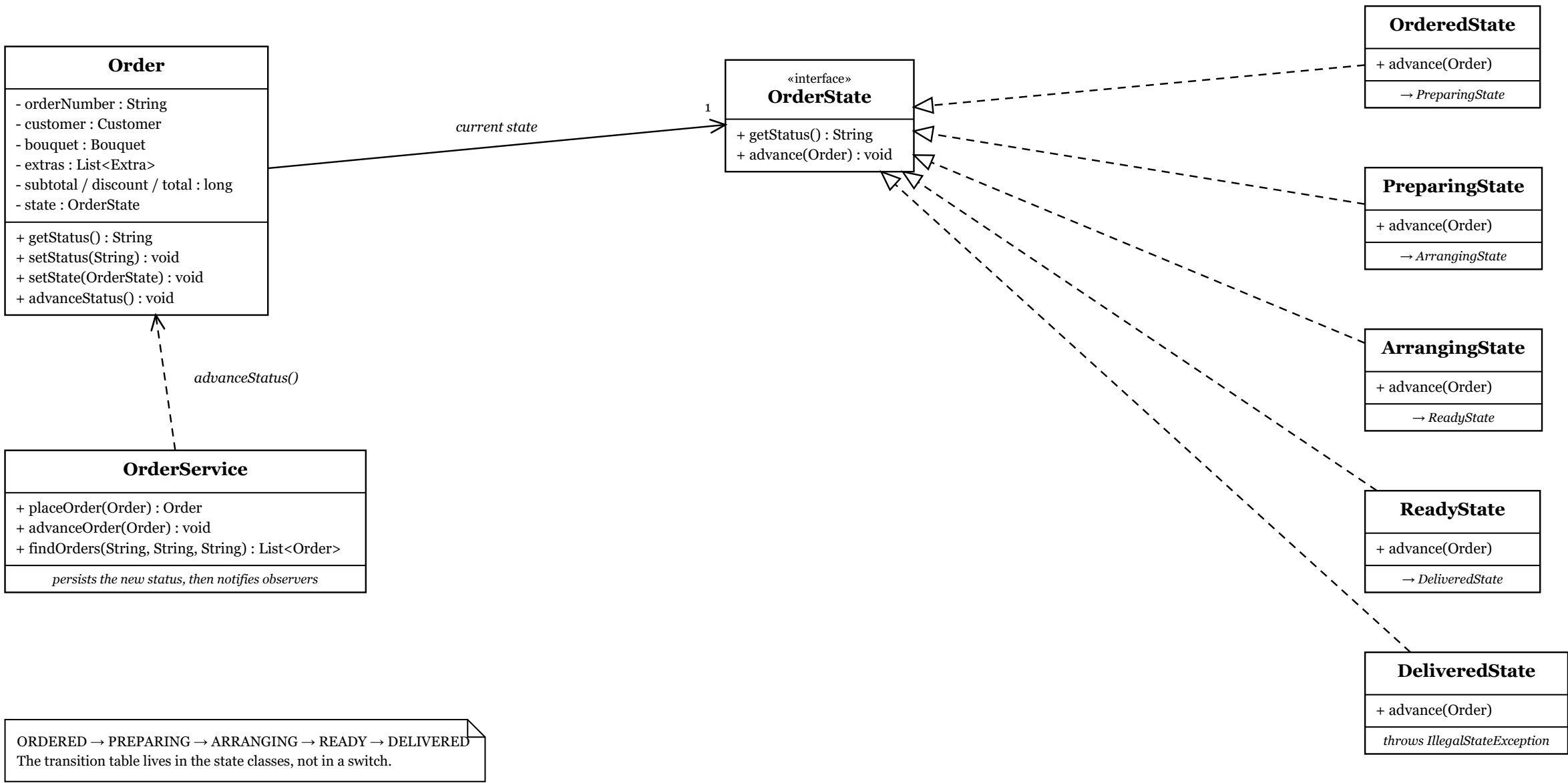


Notation:

solid line = association · open arrowhead = navigable · hollow diamond = aggregation
dashed line + hollow triangle = implements · solid line + hollow triangle = extends
dashed line + open arrowhead = uses (dependency) · italic type name = abstract

4. State — order lifecycle

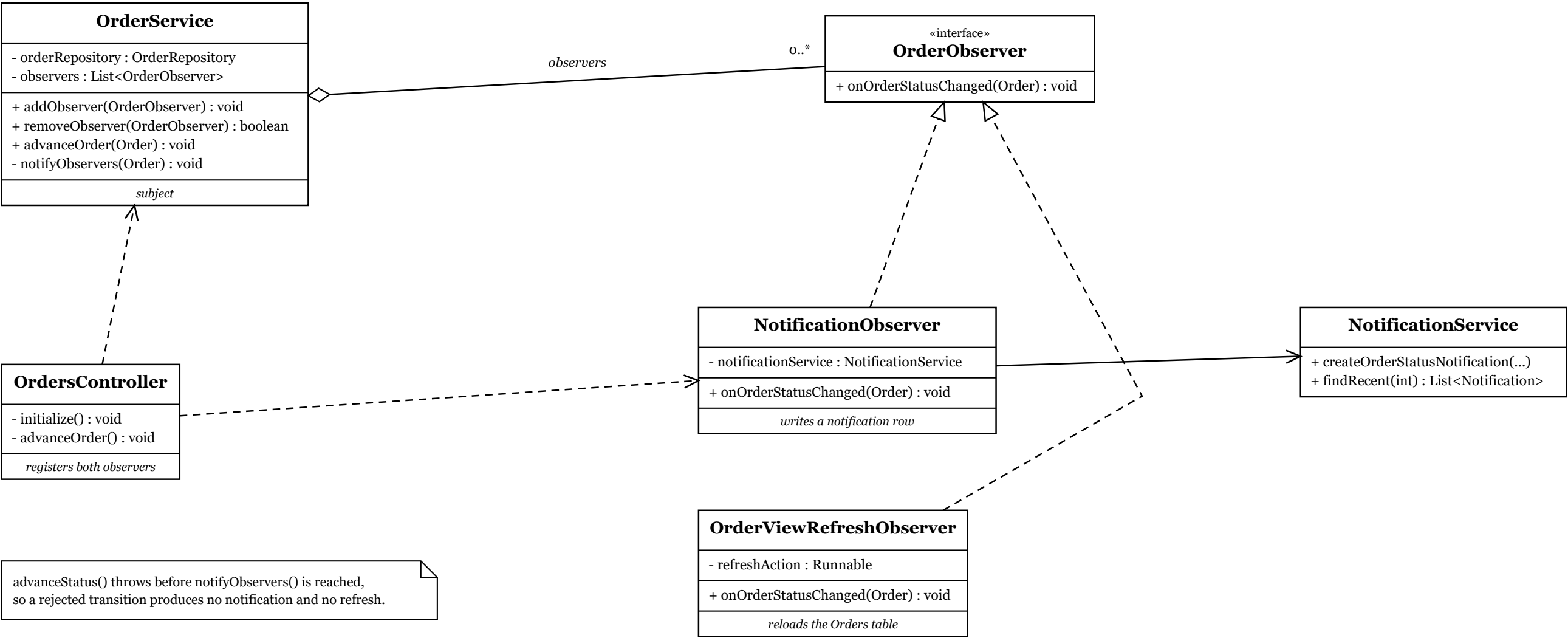
Each state knows only its own successor; *DeliveredState* refuses to advance



Notation:
solid line = association · open arrowhead = navigable · hollow diamond = aggregation
dashed line + hollow triangle = implements · solid line + hollow triangle = extends
dashed line + open arrowhead = uses (dependency) · italic type name = abstract

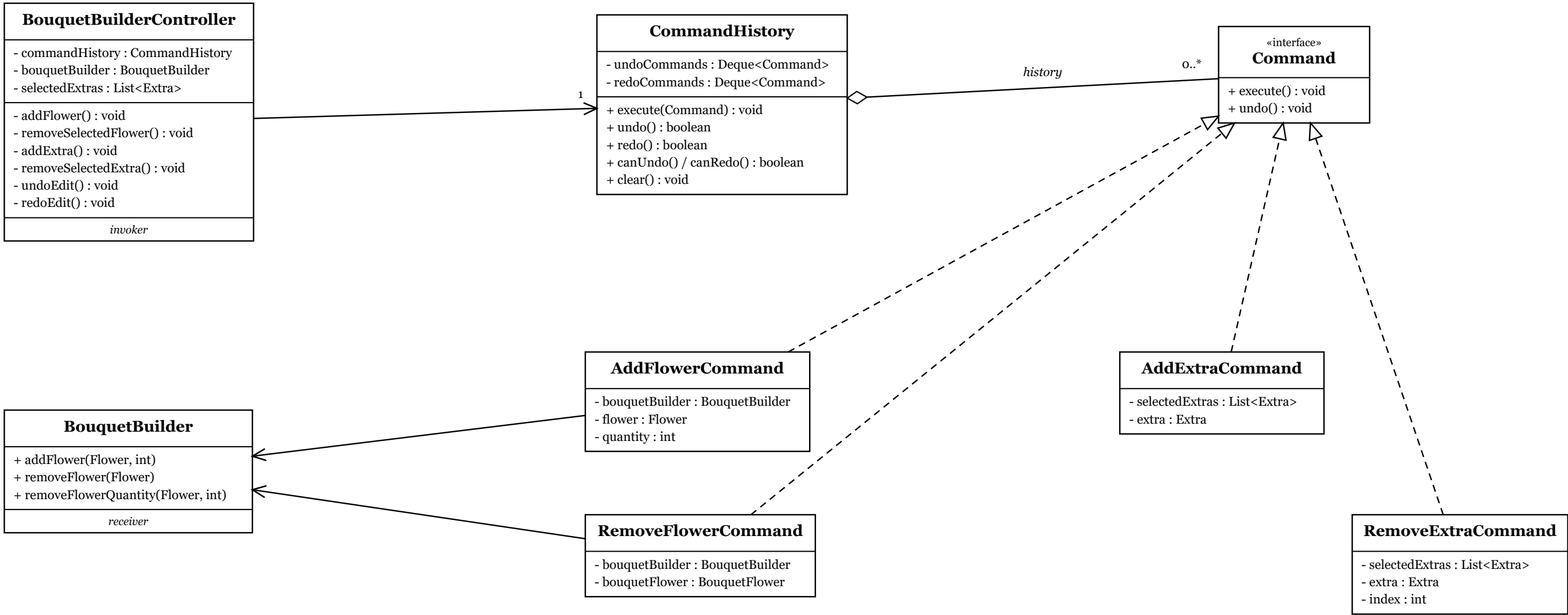
5. Observer — reactions to a successful status change

OrderService publishes the change; observers react without knowing about each other



6. Command — undo and redo in the bouquet editor

Every reversible editor action is an object with `execute()` and `undo()`

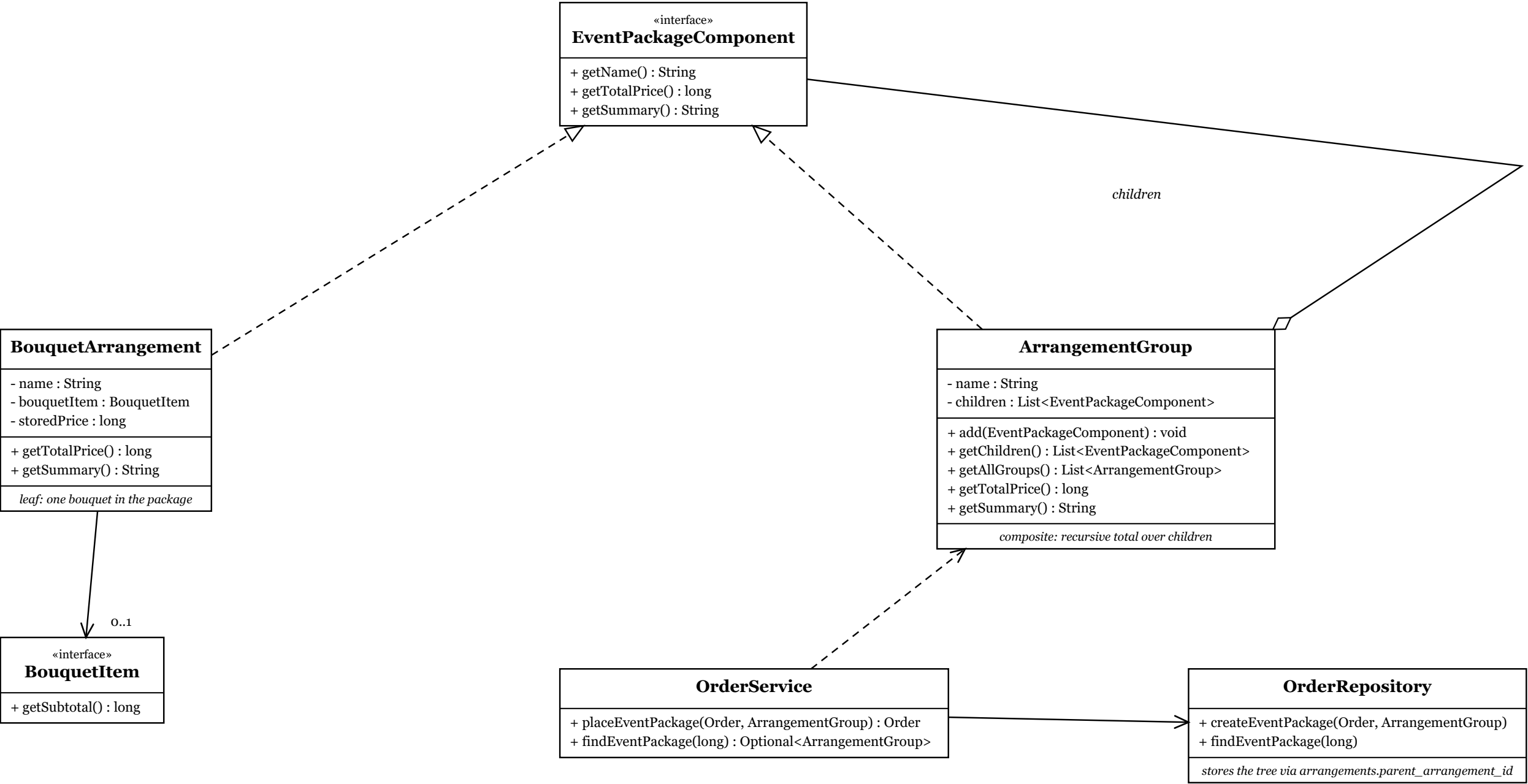


Notation:

solid line = association · open arrowhead = navigable · hollow diamond = aggregation
dashed line + hollow triangle = implements · solid line + hollow triangle = extends
dashed line + open arrowhead = uses (dependency) · italic type name = abstract

7. Composite — nested event packages

A group answers `getTotalPrice()` by asking its children, at any depth



Wedding Package
Ceremony → Bridal Bouquet, Altar Arrangement
Reception → Tables → Head Table
`getTotalPrice()` sums every leaf below the root.

Notation:
solid line = association · open arrowhead = navigable · hollow diamond = aggregation
dashed line + hollow triangle = implements · solid line + hollow triangle = extends
dashed line + open arrowhead = uses (dependency) · italic type name = abstract

8. Layered structure

controller → service → domain / pattern → repository → SQLite; dependencies point one way only

